

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC QUỐC TẾ



**BÁO CÁO ĐỒ ÁN**

MÔN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Dự án: *Plants vs Zombies: Golden Spatula* (Greenfoot Framework)

**Tác giả: Lê Anh Kiệt**

Giảng viên: L.D.Tân, N.Q.Phú

Ngày 12 tháng 5 năm 2026

## Thành viên nhóm

| STT | Họ và Tên              | MSSV        | Vai trò | Trách nhiệm chính                                            |
|-----|------------------------|-------------|---------|--------------------------------------------------------------|
| 1   | Lê Anh Kiệt            | ITCSIU24047 | All     |                                                              |
| 2   | Lê Thanh Hoàng         | ITCSIU24034 | UI      | Xây dựng các lớp con Plant, chi phí/hồi chiêu và hoạt ảnh.   |
| 3   | Nguyễn Quốc Trung Kiên | ITCSIU24034 | Zombie  | Xây dựng các lớp con Plant, cơ chế đạn và đặt cây trên lưới. |
| 4   | Dương Gia Thực         | ITITWE23026 | Zombie  | Xây dựng các lớp con Zombie, stats, hoạt ảnh và spawn wave.  |
| 5   | Nguyễn Viết Huy        | ITCSIU24034 |         |                                                              |

# Mục lục

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Introduction</b>                                                    | <b>5</b>  |
| <b>2</b> | <b>Phân tích các Mẫu thiết kế (Design Patterns) áp dụng cho Thực thể Plant</b> | <b>8</b>  |
| 2.1      | 1. Mẫu thiết kế Factory Method (Creational Pattern)                            | 8         |
| 2.2      | 2. Mẫu thiết kế Singleton (Creational Pattern)                                 | 8         |
| 2.3      | 3. Mẫu thiết kế State (Behavioral Pattern)                                     | 8         |
| 2.4      | 4. Mẫu thiết kế Observer - Event Bus (Behavioral Pattern)                      | 8         |
| 2.5      | 5. Mẫu thiết kế Strategy thông qua Composition (Behavioral Pattern)            | 9         |
| 2.6      | 6. Mẫu thiết kế Flyweight Caching (Structural Pattern)                         | 9         |
| <b>3</b> | <b>Kiến trúc các Thực thể Zombie (Zombie Entities Architecture)</b>            | <b>10</b> |
| 3.1      | 1. Lớp Trừu tượng cốt lõi (Core Abstract Zombie)                               | 10        |
| 3.2      | 2. Hệ thống Quản lý Trạng thái (Zombie State Pattern)                          | 10        |
| 3.3      | 3. Cấu hình và Khởi tạo (Configuration và Factory)                             | 10        |
| 3.4      | 4. Giao tiếp và Tài nguyên (Event Bus và Assets)                               | 11        |
| <b>4</b> | <b>Hệ thống Quản lý Ô lưới (Grid Management) và Luồng xử lý Tương tác</b>      | <b>12</b> |
| 4.1      | 1. Cấu trúc Quản lý Grid (GridManager Architecture)                            | 12        |
| 4.2      | 2. Luồng xử lý tương tác kéo thả (Placement Sequence)                          | 12        |
| 4.3      | 3. Mẫu thiết kế áp dụng trong Hệ thống Grid                                    | 12        |
| <b>5</b> | <b>Kiến trúc hệ thống SeedBank và Quản lý Thẻ bài (SeedPacket)</b>             | <b>14</b> |
| 5.1      | 1. Quản lý kho thẻ bài (SeedBank)                                              | 14        |
| 5.2      | 2. Trạng thái thẻ bài (SeedPacket State Pattern)                               | 14        |
| 5.3      | 3. Quy trình khởi tạo thực thể thông qua Factory                               | 14        |
| 5.4      | 4. Điều khiển kéo thả (DragController)                                         | 14        |
| <b>6</b> | <b>Kiến trúc hệ thống Shovel và Cơ chế Giải phóng thực thể (Plant Removal)</b> | <b>16</b> |
| 6.1      | 1. Cấu trúc của công cụ Shovel (Shovel Architecture)                           | 16        |
| 6.2      | 2. Bộ điều khiển hành động Đào cây (SellShovel Controller)                     | 16        |
| 6.3      | 3. Luồng tương tác giữa Shovel và Môi trường                                   | 16        |
| 6.4      | 4. Các nguyên lý thiết kế áp dụng                                              | 17        |
| <b>7</b> | <b>Kiến trúc hệ thống Roll và Cơ chế Xác suất (Gacha Logic)</b>                | <b>18</b> |
| 7.1      | 1. Bộ điều phối nâng cấp cấp độ (RupButton)                                    | 18        |
| 7.2      | 2. Cơ chế Roll thẻ bài dựa trên xác suất (RollButton)                          | 18        |
| 7.3      | 3. Mối liên hệ với PlayScene và SunManager                                     | 18        |
| 7.4      | 4. Mẫu thiết kế áp dụng                                                        | 18        |
| <b>8</b> | <b>Kiến trúc hệ thống Augment và Điều phối đợt tấn công (Wave Management)</b>  | <b>20</b> |
| 8.1      | 1. Thực thể Thẻ nâng cấp (AugmentCard)                                         | 20        |
| 8.2      | 2. Điều phối vòng lặp trò chơi (WaveManager Integration)                       | 20        |
| 8.3      | 3. Sự phối hợp đa hệ thống (System Coordination)                               | 20        |
| 8.4      | 4. Mẫu thiết kế áp dụng                                                        | 21        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>9</b> | <b>Hệ thống Quản lý Tài nguyên (Sun Management System)</b> | <b>22</b> |
| 9.1      | 1. Bộ điều phối sản sinh tài nguyên (SunSpawner) . . . . . | 22        |
| 9.2      | 2. Bộ quản lý ngân sách trung tâm (SunManager) . . . . .   | 22        |
| 9.3      | 3. Thực thể Tài nguyên động (FallingSun) . . . . .         | 22        |
| 9.4      | 4. Mẫu thiết kế áp dụng trong Module Tài nguyên . . . . .  | 23        |

# 1 Project Introduction

This project delivers a sophisticated gameplay experience by synthesizing the classic tower-defense mechanics of *Plants vs. Zombies* with the strategic depth of the "Auto-battler" genre, popularized by *Teamfight Tactics (TFT)*. Rather than adhering to traditional static lane placements, players are immersed in a dynamic tactical arena where environmental adaptation and strategic resource management are paramount.

- **A Unique Systemic Convergence:** While preserving the iconic visual identity of the original flora and undead entities, the game operates on a **hexagonal grid system**. This architectural shift from linear paths to multi-dimensional combat unlocks an extensive spectrum of creative formations and strategic maneuvers.
- **High-Stakes Strategic Mechanics:**
  - **Randomized Procurement:** Players must strategically allocate their *Sun* resources to "Roll" and navigate an ever-changing recruitment pool, scouting for legendary units or the essential components required to complete a specific synergy.
  - **Evolutionary Ascension:** Upon collecting identical units, the system automatically triggers a star-level upgrade. This mechanic facilitates a dual progression of both **aesthetic evolution** and **enhanced combat efficacy**.

## 2. Game Overview

The game is played on a fixed grid where players strategically place plants to defend against incoming waves of zombies. The core mechanics involve resource management and tactical placement.

### Core Mechanics

- **Grid Placement:** Plants are placed onto the grid, with each occupying a single cell. Placement requires spending “sun,” the primary in-game currency.
- **Sun System:** Sun resources drop randomly on the screen and must be collected by the player to be added to their total. Certain plants (e.g., *Sunflower*) generate sun periodically.
- **Waves and Lanes:** Zombies spawn in timed waves and travel along fixed horizontal lanes towards the player’s house.
- **Projectiles and Collision:** Plants attack using projectiles (e.g., *PeaProjectile*, *FrozenPeaProjectile*), which travel across the lane and deal damage upon collision with a zombie. Collision checks are performed during the update loop.
- **Game Over Condition:** The game ends when a zombie successfully breaches the last column of the grid and reaches the player’s house, or if the special defense item (*Lawn Mower*) for that lane has already been consumed.

### Visual Overview

[PLACEHOLDER: Insert Game Screenshot Here]

*Figure 1: Main Gameplay Interface showing Grid and Sun Economy*

[PLACEHOLDER: Insert Projectile/Collision Diagram Here]

*Figure 2: Visualization of Lane-based Movement and Collision Detection*

## 3. Architecture

The project utilizes an **Entity-Component-System (ECS)**-inspired approach to decouple data (*Components*) from logic (*Systems*). Entities are simple containers holding various Components, while Systems operate on Entities that possess specific Component sets.

### 3.1. Packages

| Package                     | Key Classes                                                     | Responsibilities                                                                                    |
|-----------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| pvz.com.screens             | GameScreen,<br>GameOverScreen                                   | Manage the display and control flow for different stages of the game (main game loop vs. game end). |
| pvz.com.logic               | GameWorld,<br>GameState,<br>ZombieWaveController,<br>...        | High-level game coordination, state management, timing, and control of non-entity game elements.    |
| pvz.com.entities            | Entity,      Plant,<br>Zombies,      Sun,<br>Projectile         | Base classes for all game objects and their hierarchical specializations.                           |
| pvz.com.entities.components | Position,      Bounds,<br>Health,      Animation,<br>State, ... | Data containers defining an Entity's attributes and state.                                          |
| pvz.com.systems             | Render,      Movement,<br>Collision,      Cleanup,<br>...       | Core logic modules that process Entities based on their Components.                                 |

Bảng 1: Package structure and responsibilities.

### 3.2. Interfaces

To ensure loose coupling and follow the **Dependency Inversion Principle**, the architecture relies on the following key interfaces:

- **IGameSpawner:** Interface used by controllers (e.g., `ZombieWaveController`) to abstract the creation and placement of new entities into the `GameWorld`. This allows systems to spawn entities without knowing the concrete implementation of the world.
- **ISunReceiver:** Interface implemented by classes that need to process collected sun resources (e.g., `HudController` or `GameState`). It decouples the sun collection logic from the UI update logic.

## 2 Phân tích các Mẫu thiết kế (Design Patterns) áp dụng cho Thực thể Plant

Hệ thống thực thể Plant trong dự án không chỉ kế thừa các đặc tính vật lý và đồ họa mà còn tích hợp nhiều mẫu thiết kế kinh điển để giải quyết các vấn đề về quản lý trạng thái, khởi tạo đối tượng và tối ưu hóa tài nguyên.

### 2.1 1. Mẫu thiết kế Factory Method (Creational Pattern)

Hệ thống sử dụng `PlantFactory` hiện thực interface `IPlantFactory` để quản lý việc tạo mới các thực thể.

- **Mục đích:** Tách biệt logic khởi tạo phức tạp (tạo `Plant` hoặc `SeedPacket`) khỏi logic điều khiển trò chơi.
- **Cách thức:** Thay vì gọi trực tiếp constructor của từng lớp cụ thể, hệ thống gọi phương thức `createPlant(PlantType type)`. Điều này cho phép dễ dàng thêm các loại cây mới vào Enum `PlantType` mà không cần thay đổi mã nguồn ở những nơi sử dụng.

### 2.2 2. Mẫu thiết kế Singleton (Creational Pattern)

Lớp `PlantFactory` được thiết kế theo mẫu Singleton với phương thức `static getInstance()`.

- **Mục đích:** Đảm bảo toàn bộ ứng dụng chỉ tồn tại một thực thể nhà máy duy nhất, giúp kiểm soát tập trung việc cấp phát tài nguyên và quản lý các loại thực thể cây được phép tạo ra.

### 2.3 3. Mẫu thiết kế State (Behavioral Pattern)

Sự phối hợp giữa `Plant`, `PlantStateManager` và `PlantState` (Enum) minh chứng cho việc áp dụng mẫu thiết kế State.

- **Mục đích:** Thay đổi hành vi của cây một cách linh hoạt dựa trên tình trạng hiện tại mà không cần sử dụng các cấu trúc `if-else` hay `switch-case` cồng kềnh trong hàm `act()`.
- **Trạng thái cụ thể:** Thực thể tự động chuyển đổi giữa các trạng thái `IDLE` (chờ), `ATTACKING` (tấn công), `DYING` (đang chết) và `DRAGGING` (đang bị kéo). Mỗi trạng thái sẽ quy định cách cây phản ứng với môi trường và cách hiển thị hoạt ảnh.

### 2.4 4. Mẫu thiết kế Observer - Event Bus (Behavioral Pattern)

Hệ thống sử dụng `PlantEventBus` đóng vai trò là "Subject" và `IPlantEventListener` là "Observer".

- **Mục đích:** Tạo ra cơ chế liên lạc lỏng lẻo (*Loose Coupling*) giữa thực thể cây và các thành phần khác như UI hoặc hệ thống âm thanh.
- **Ứng dụng:** Khi một cây thực hiện hành động (như `publishMerge` hoặc `publishDeath`), nó chỉ cần phát sự kiện lên `EventBus`. Các lớp quan tâm (Subscribers) sẽ tự động nhận thông báo và cập nhật (ví dụ: trừ tiền, phát tiếng nổ) mà cây không cần biết lớp đó là gì.



## 2.5 5. Mẫu thiết kế Strategy thông qua Composition (Behavioral Pattern)

Thay vì dồn toàn bộ logic vào lớp cha `Plant`, kiến trúc sử dụng các lớp Handler chuyên biệt:

- **PlantInputHandler:** Chiến lược xử lý tương tác người dùng.
- **PlantCombatHandler:** Chiến lược xử lý tính toán sát thương và va chạm.
- **Lợi ích:** Cho phép thay đổi thuật toán xử lý (ví dụ: đổi cách cây nhận sát thương) bằng cách thay thế lớp Handler tương ứng mà không phải thay đổi cấu trúc phân cấp kế thừa của cây.

## 2.6 6. Mẫu thiết kế Flyweight Caching (Structural Pattern)

Được thể hiện qua lớp `SpriteCache` được sử dụng bởi `SpriteAnimator`.

- **Mục đích:** Tối ưu hóa bộ nhớ khi có hàng chục thực thể cùng loại trên màn hình.
- **Cơ chế:** Sử dụng một `Map<String, GreenfootImage[]>` để lưu trữ dùng chung các tập tin hình ảnh hoạt ảnh. Các thực thể cùng loại (ví dụ: 10 cây Peashooter) sẽ cùng truy cập vào một vùng nhớ ảnh duy nhất thay vì mỗi cây tự nạp một bộ ảnh riêng.

### 3 Kiến trúc các Thực thể Zombie (Zombie Entities Architecture)

Dựa trên sơ đồ lớp (UML Diagram), hệ thống Zombie được thiết kế để quản lý các thực thể kẻ thù với độ phức tạp cao, tập trung vào việc tách biệt giữa dữ liệu cấu hình, trạng thái hành vi và xử lý sự kiện.

#### 3.1 1. Lớp Trừu tượng cốt lõi (Core Abstract Zombie)

Lớp `Zombie` kế thừa từ `PhysicsBody` là xương sống cho mọi loại kẻ thù trong trò chơi. Các đặc điểm chính bao gồm:

- **Quản lý Chỉ số:** Lưu trữ các thuộc tính quan trọng như `hp`, `maxHp`, và `walkSpeed`. Lớp này cung cấp các phương thức điều khiển vòng đời như `hit(int dmg)`, `isLiving()` và `deathAnim()`.
- **Logic Tương tác:** Phương thức `checkEating()` cho phép Zombie kiểm tra và tương tác với các thực thể `IEatable` (thường là `Plants`) trên đường đi.
- **Hệ thống Animation:** Tách biệt các bộ khung hình cho các tình huống khác nhau như `headless` (khi mất đầu), `headlesseating`, và `fall` (khi chết).

#### 3.2 2. Hệ thống Quản lý Trạng thái (Zombie State Pattern)

Hệ thống sử dụng mẫu thiết kế `**State**` thông qua interface `IZombieState` để quản lý hành vi phức tạp:

- **WalkingState:** Định nghĩa hành vi di chuyển cơ bản của Zombie dựa trên `walkSpeed`.
- **EatingState:** Kích hoạt khi Zombie tiếp cận được mục tiêu, chuyển đổi từ di chuyển sang tấn công định kỳ.
- **DeadState:** Xử lý logic sau khi chết, thực hiện hoạt ảnh rơi rụng và gọi hàm `cleanup()` để giải phóng tài nguyên.
- **Cơ chế chuyển đổi:** Phương thức `setState(IZombieState newState)` cho phép thực thể thay đổi hành vi tức thời khi điều kiện môi trường thay đổi.

#### 3.3 3. Cấu hình và Khởi tạo (Configuration và Factory)

Để tối ưu hóa việc tạo ra nhiều loại Zombie khác nhau, hệ thống áp dụng cơ chế nạp dữ liệu tách biệt:

- **ZombieConfig:** Một lớp đóng gói dữ liệu (Data Class) chứa các thông số như máu, sát thương, tốc độ và các ngưỡng trạng thái (`thresholds`). Các hằng số như `BASIC`, `CONE`, `RA` được định nghĩa sẵn để tái sử dụng.
- **ZombieFactory:** Sử dụng mẫu thiết kế `**Factory**` để khởi tạo các thực thể Zombie dựa trên `ZombieType` hoặc tên định danh, giúp đóng gói logic thiết lập phức tạp.
- **ZombieRegistry:** Đóng vai trò như một kho lưu trữ tập trung các hằng số hệ thống (ví dụ: `BASIC_HP`, `RA_SPEED`).

### 3.4 4. Giao tiếp và Tài nguyên (Event Bus và Assets)

- **ZombieEventBus:** Hiện thực mẫu thiết kế **\*\*Observer\*\***, cho phép phát các sự kiện (`publishHit`, `publishDeath`, `publishAteTarget`) đến các đối tượng lắng nghe (`IZombieEventListener`) mà không làm tăng độ phụ thuộc giữa các lớp.
- **ZombieAssets & SpriteCache:** Quản lý việc nạp tài nguyên hình ảnh theo cơ chế **\*\*Flyweight\*\***. Hình ảnh được tải từ đường dẫn (`path`) và lưu trữ trong **SpriteCache** để chia sẻ giữa hàng loạt thực thể cùng loại, giảm thiểu tải trọng cho bộ nhớ RAM.

*Nhận xét kỹ thuật:* Việc tách biệt hoàn toàn **ZombieConfig** ra khỏi logic của lớp **Zombie** giúp hệ thống cực kỳ linh hoạt. Người phát triển có thể cân bằng lại game (re-balance) chỉ bằng cách thay đổi thông số trong Registry hoặc Config mà không cần can thiệp vào logic xử lý di chuyển hay tấn công.

## 4 Hệ thống Quản lý Ô lưới (Grid Management) và Luồng xử lý Tương tác

Hệ thống ô lưới đóng vai trò là nền tảng logic để định vị, quản lý không gian và điều phối các tương tác giữa thực thể Plant và môi trường trò chơi.

### 4.1 1. Cấu trúc Quản lý Grid (GridManager Architecture)

Lớp `GridManager` hiện thực interface `IPlantPlacer`, đóng vai trò là bộ điều phối trung tâm cho toàn bộ bàn chơi.

- **Quản lý Không gian:** Sử dụng mảng hai chiều `Plant[][] Board` để lưu trữ trạng thái của từng ô lưới. Các hằng số `ROWS`, `COLS` và `HEX_R` định nghĩa kích thước hình học và logic của bàn chơi.
- **Kiểm soát Khả năng Đặt cây:** Phương thức `canPlace(int gx, int gy, Plant plant)` thực hiện kiểm tra điều kiện logic (ô trống, đủ điều kiện cấp độ) trước khi cho phép đặt thực thể vào vị trí đích.
- **Cơ chế Tự động hóa:** Phương thức `autoCheckAllCombines()` cho phép hệ thống quét toàn bộ bàn chơi để kích hoạt các logic hợp nhất thực thể một cách tự động khi điều kiện thỏa mãn.

### 4.2 2. Luồng xử lý tương tác kéo thả (Placement Sequence)

Dựa trên Sequence Diagram, quy trình đặt một Plant vào thế giới diễn ra theo các bước nghiêm ngặt nhằm đảm bảo tính toàn vẹn dữ liệu:

1. **Sự kiện Người dùng:** Người chơi thực hiện hành động "Thả Plant" tại tọa độ chuột  $(mx, my)$ .
2. **Chuyển đổi Tọa độ:** `GridManager` gọi hàm nội bộ `getGridPos(mx, my)` để ánh xạ từ tọa độ màn hình sang chỉ số ô lưới  $(gx, gy)$ .
3. **Kiểm tra Điều kiện (alt logic):**
  - **Trường hợp True:** Nếu `canPlace()` trả về `True`, hệ thống cập nhật `Board[gy][gx] = plant`, thiết lập vị trí thực thể qua `setLocation()` và kích hoạt `checkAndCombine()` thông qua `PlantCombineHandler`.
  - **Trường hợp False:** Nếu vị trí không hợp lệ, hệ thống gọi `returnBackToSafePos(plant)` để trả thực thể về vị trí cũ hoặc hàng chờ (Row 5), ngăn chặn lỗi đè dữ liệu.

### 4.3 3. Mẫu thiết kế áp dụng trong Hệ thống Grid

- **Strategy Pattern:** Việc `GridManager` hiện thực `IPlantPlacer` cho phép linh hoạt thay đổi thuật toán đặt cây (ví dụ: đặt trên ô vuông truyền thống hoặc ô lục giác) mà không ảnh hưởng đến các lớp gọi.
- **Mediator Pattern:** `GridManager` hoạt động như một trung gian điều phối giữa `Player (Mouse)`, thực thể `Plant` và `PlantCombineHandler`, giúp giảm thiểu sự phụ thuộc trực tiếp giữa các thành phần này.

- **Delegation:** Logic xử lý việc kết hợp cây sau khi đặt được ủy thác (*delegate*) cho lớp chuyên trách `PlantCombineHandler`, tuân thủ nguyên lý Đơn nhiệm (Single Responsibility).

*Nhận xét kỹ thuật:* Cơ chế kiểm tra `alt` trong luồng xử lý đảm bảo tính ổn định của Game Loop. Việc tách biệt giữa tọa độ thực (*pixel*) và tọa độ logic (*grid index*) giúp việc tính toán va chạm và xử lý hoạt ảnh di chuyển của `Merger` trở nên chính xác hơn.

## 5 Kiến trúc hệ thống SeedBank và Quản lý Thẻ bài (SeedPacket)

Hệ thống SeedBank đóng vai trò là giao diện điều khiển trung tâm (UI Controller), quản lý việc lựa chọn và thực hiện mua các thực thể thực vật thông qua các gói hạt giống (SeedPackets).

### 5.1 1. Quản lý kho thẻ bài (SeedBank)

Lớp SeedBank đóng vai trò là container quản lý danh sách các SeedPacket.

- **Thành phần:** Chứa một mảng các bank kiểu SeedPacket và một DragController để xử lý logic kéo thả cây từ thẻ bài ra môi trường thế giới.
- **Cơ chế cập nhật:** Phương thức updateBank() cho phép thay đổi danh sách thẻ bài tùy theo tiến trình màn chơi, trong khi placePacketsInWorld() chịu trách nhiệm render các thẻ bài lên giao diện người dùng.

### 5.2 2. Trạng thái thẻ bài (SeedPacket State Pattern)

Để xử lý logic thẻ bài (như khi đủ tiền mua, khi đang trong thời gian hồi (cooldown), hoặc khi đang được chọn), hệ thống áp dụng mẫu thiết kế **\*\*State\*\*** thông qua interface IPacketState:

- **AvailableState:** Trạng thái thẻ bài sẵn sàng để tương tác khi người chơi có đủ điểm mặt trời (sunCost).
- **SelectedState:** Trạng thái khi người chơi đã click chọn thẻ bài, kích hoạt DragController để chuẩn bị đặt cây.
- **Tính linh hoạt:** Việc tách biệt trạng thái giúp phương thức onSunChanged() của SeedPacket tự động cập nhật hiển thị (sáng lên hoặc tối đi) dựa trên số lượng sun hiện có mà không làm phức tạp logic điều khiển.

### 5.3 3. Quy trình khởi tạo thực thể thông qua Factory

Sơ đồ UML thể hiện mối quan hệ chặt chẽ giữa hệ thống thẻ bài và PlantFactory (hiện thực IPlantFactory):

- **Khởi tạo gián tiếp:** SeedBank yêu cầu PlantFactory tạo các đối tượng SeedPacket thông qua phương thức createSeedPacket(String type).
- **Sinh thực thể:** Khi một SeedPacket được sử dụng thành công, nó sẽ gửi yêu cầu (request) đến PlantFactory để tạo thực thể Plant tương ứng (như Peashooter, Repeater) để đặt vào Grid.

### 5.4 4. Điều khiển kéo thả (DragController)

DragController hoạt động như một lớp trung gian (Mediator) giữa UI và Game World:

- Nó nhận thông tin từ thẻ bài đang chọn và phối hợp với SunDisplay để kiểm tra điều kiện thanh toán.

- Sử dụng interface `IPlantPlacer` (ẩn trong logic xử lý) để xác định vị trí hợp lệ trên ô lưới trước khi chính thức gọi `Factory` tạo cây.

*Nhận xét kỹ thuật:* Kiến trúc này tách biệt hoàn toàn giữa "vật chứa dữ liệu mua bán"(`SeedPacket`) và "thực thể chiến đấu thực sự"(`Plant`). Điều này cho phép hệ thống quản lý tài nguyên hiệu quả, chỉ khởi tạo thực thể `Plant` khi thực sự cần thiết, giúp tối ưu hóa hiệu suất cho framework `Greenfoot`.

## 6 Kiến trúc hệ thống Shovel và Cơ chế Giải phóng thực thể (Plant Removal)

Hệ thống Shovel được thiết kế để cung cấp cho người chơi công cụ tương tác trực tiếp nhằm loại bỏ các thực thể Plant hiện có trên bàn chơi, giải phóng ô lưới và thực hiện các logic bổ trợ (như hoàn tiền hoặc tái cấu trúc đội hình).

### 6.1 1. Cấu trúc của công cụ Shovel (Shovel Architecture)

Lớp Shovel kế thừa từ `SpriteAnimator`, đóng vai trò là một đối tượng UI có khả năng tương tác cao.

- **Quản lý hiển thị:** Sử dụng hai tài nguyên hình ảnh `IMG_NORMAL` và `IMG_SELECTED` để phản hồi trạng thái tương tác của người chơi.
- **Trạng thái lựa chọn:** Thuộc tính `selected` xác định liệu công cụ có đang được người chơi kích hoạt để sử dụng hay không thông qua phương thức `setSelected()`.

### 6.2 2. Bộ điều khiển hành động Đào cây (SellShovel Controller)

Khi người chơi kích hoạt xẻng, hệ thống sẽ sinh ra (*spawns*) một thực thể `SellShovel`. Đây là một "Action Controller" chịu trách nhiệm xử lý logic đào cây phức tạp:

- **Theo dõi vị trí:** Lưu trữ `startX` và `startY` để quản lý vị trí ban đầu của xẻng trên UI.
- **Phản hồi thị giác (Highlighting):** Phương thức `updateHighlight(x, y)` thực hiện việc làm nổi bật thực thể `Plant` đang nằm dưới con trỏ xẻng, giúp người chơi xác định chính xác mục tiêu cần xóa.
- **Thực thi hành động (Digging):** Khi người chơi thả xẻng (`handleDrop`), phương thức `dig(gx, gy)` sẽ được gọi để truy vấn `GridManager` và thực hiện việc gỡ bỏ thực thể tại ô lưới tương ứng.

### 6.3 3. Luồng tương tác giữa Shovel và Môi trường

Dựa trên mối quan hệ giữa các lớp, quy trình đào cây diễn ra thông qua sự phối hợp của 3 thành phần chính:

1. **SellShovel và PlayScene:** Bộ điều khiển tham chiếu đến `PlayScene` để truy cập vào `gridManager`, từ đó xác định tọa độ logic của các ô đất.
2. **SellShovel và Plant:** Bộ điều khiển giữ một tham chiếu `lastHighlighted` để quản lý thực thể đang bị tác động, đồng thời thay đổi thuộc tính `opaque` của cây để tạo hiệu ứng mờ trước khi thực hiện lệnh xóa.
3. **Hoàn tất hành động:** Sau khi thực hiện xong lệnh `dig`, bộ điều khiển gọi hàm `exit()` để tự hủy và đưa xẻng về vị trí gốc trên thanh công cụ.



## 6.4 4. Các nguyên lý thiết kế áp dụng

- **Command Pattern:** Lớp `SellShovel` hoạt động tương tự như một đối tượng `Command`, đóng gói toàn bộ yêu cầu "xóa cây" và các tham số liên quan vào một thực thể độc lập để quản lý vòng đời hành động.
- **Visual Feedback Strategy:** Việc tách biệt hình ảnh `NORMAL` và `SELECTED` cùng với logic `updateHighlight` cung cấp trải nghiệm UX mượt mà, giúp người chơi tránh các thao tác sai lầm trong lúc gameplay căng thẳng.

*Nhận xét kỹ thuật:* Việc sử dụng một lớp trung gian `SellShovel` để xử lý logic thay vì viết trực tiếp vào lớp `Shovel` giúp tách biệt giữa thành phần giao diện (Static UI) và thành phần xử lý logic hành động (Dynamic Action logic), tuân thủ tốt nguyên lý **Separation of Concerns**.

## 7 Kiến trúc hệ thống Roll và Cơ chế Xác suất (Gacha Logic)

Hệ thống cửa hàng và nâng cấp được thiết kế theo mô hình điều khiển sự kiện, kết hợp với các thuật toán xác suất để tạo ra trải nghiệm người chơi đa dạng thông qua cơ chế "Roll" và nâng cấp cấp độ người chơi (Level Up).

### 7.1 1. Bộ điều phối nâng cấp cấp độ (RupButton)

Lớp `RupButton` đóng vai trò là giao diện tương tác để người chơi nâng cấp chỉ số hệ thống.

- **Quản lý cấp độ:** Lưu trữ `currentLevel` và `MAX_LEVEL`. Khi người chơi thực hiện nâng cấp, chi phí `UPGRADE_COST` sẽ được khấu trừ thông qua `SunManager`.
- **Cập nhật Pool tài nguyên:** Mỗi khi `currentLevel` tăng lên, `weightedPool` (danh sách các thẻ bài có thể xuất hiện) sẽ được cập nhật lại. Điều này cho phép các thẻ bài hiếm (`Rarity`) xuất hiện với tỉ lệ cao hơn ở cấp độ cao.

### 7.2 2. Cơ chế Roll thẻ bài dựa trên xác suất (RollButton)

Lớp `RollButton` thực hiện logic cốt lõi của cơ chế Auto-battler:

- **Thuật toán xác suất có trọng số (Weighted Probability):** Sử dụng lớp hỗ trợ `RarityEntry` chứa cặp giá trị `<Class, weight>`. Phương thức `generateValidPacket()` sẽ tính tổng trọng số và thực hiện "quay số" để chọn ra thẻ bài ngẫu nhiên từ Pool.
- **Khởi tạo qua Reflection:** Một điểm đặc biệt là hệ thống sử dụng *Instantiates via Reflection* để tạo ra các đối tượng `SeedPacket` từ lớp (`Class`) được định nghĩa trong Pool. Điều này giúp tối ưu hóa việc mở rộng: bạn có thể thêm cây mới vào Pool mà không cần sửa code của `RollButton`.

### 7.3 3. Mối liên hệ với PlayScene và SunManager

Sơ đồ thể hiện sự phụ thuộc chặt chẽ giữa logic mua bán và tài nguyên hệ thống:

- **Truy vấn tài nguyên:** Cả `RupButton` và `RollButton` đều gọi `getSunManager()` từ `PlayScene` để thực hiện các phương thức `hasEnough()` và `spend()`.
- **Đồng bộ trạng thái:** Sau mỗi lần thực hiện lệnh `executeRoll()` hoặc `executeUpgrade()`, phương thức `updateAppearance()` sẽ được gọi để cập nhật hiển thị (nút bấm sáng/tối hoặc đổi màu) phản hồi cho người chơi.

### 7.4 4. Mẫu thiết kế áp dụng

- **Strategy Pattern:** Cơ chế xác suất trong `weightedPool` có thể coi là một chiến lược chọn lọc tài nguyên. Bằng cách thay đổi danh sách `RarityEntry`, hệ thống thay đổi hoàn toàn "chiến thuật" phân phối thẻ bài mà không làm ảnh hưởng đến luồng chạy của nút bấm.
- **Mediator Pattern:** `PlayScene` đóng vai trò trung gian, kết nối `RupButton`, `RollButton` với các hệ thống hạ tầng như `SunManager` và `SeedBank`.

*Nhận xét kỹ thuật:* Việc sử dụng mảng trọng số (`weightedPool`) kết hợp với cấp độ người chơi là một cách tiếp cận chuyên nghiệp để tạo ra sự tiến triển (Progression) trong game. Cơ chế này đảm bảo tính cân bằng (Game Balance) và tạo ra động lực để người chơi tích lũy tài nguyên cho những đợt Roll bậc cao.

## 8 Kiến trúc hệ thống Augment và Điều phối đợt tấn công (Wave Management)

Hệ thống Augment được thiết kế như một cơ chế thưởng (Reward System) cho người chơi giữa các đợt tấn công, cho phép can thiệp sâu vào chỉ số hệ thống và chiến thuật thông qua các hiệu ứng đặc biệt.

### 8.1 1. Thực thể Thẻ nâng cấp (AugmentCard)

Lớp `AugmentCard` là thành phần tương tác chính trong quá trình chọn lựa nâng cấp.

- **Quản lý tương tác:** Sử dụng biến `hovered` và phương thức `handleHover()` để phản hồi trực quan khi người chơi di chuyển chuột qua thẻ bài.
- **Định nghĩa loại nâng cấp:** Thuộc tính `type` kiểu `String` xác định loại hiệu ứng mà thẻ bài sẽ mang lại (ví dụ: tăng sát thương, giảm hồi chiêu, hoặc tăng ô chứa cây).
- **Thực thi hiệu ứng:** Phương thức `applyAugmentEffect()` chịu trách nhiệm kích hoạt logic thay đổi chỉ số hệ thống ngay khi người chơi xác nhận lựa chọn.

### 8.2 2. Điều phối vòng lặp trò chơi (WaveManager Integration)

`AugmentCard` giữ một tham chiếu trực tiếp đến `WaveManager` để điều khiển tiến trình của màn chơi:

- **Ngắt nhịp độ:** Hệ thống Augment thường xuất hiện sau khi một đợt Zombie kết thúc. Sau khi người chơi chọn xong, thẻ bài sẽ gọi `manager.nextWave()` để tiếp tục vòng lặp trò chơi.
- **Dọn dẹp giao diện:** Phương thức `clearUI()` kết hợp với `Overlay` để đóng cửa sổ chọn nâng cấp và trả lại không gian cho màn hình chiến đấu.

### 8.3 3. Sự phối hợp đa hệ thống (System Coordination)

Dựa trên sơ đồ mối quan hệ, `AugmentCard` đóng vai trò là "ngòi nổ" cho nhiều thay đổi trên toàn hệ thống:

- **Tác động đến Scene:** Gọi `increasePlantSlots()` trong `PlayScene` để mở rộng giới hạn đơn vị trên Grid.
- **Tương tác với Factory:** Có khả năng kích hoạt `PlantFactory` để sinh ra (*spawns*) các thực thể cây đặc biệt như phần thưởng trực tiếp từ Augment.
- **Quản lý hiển thị:** Sử dụng `Overlay` để làm mờ phong nền, giúp người chơi tập trung vào việc lựa chọn mà không bị xao nhãng bởi các thực thể khác đang hoạt động.

## 8.4 4. Mẫu thiết kế áp dụng

- **Command Pattern:** Mỗi `AugmentCard` đóng gói một lệnh thực thi hiệu ứng. Khi được click, nó giải phóng một chuỗi các hành động thay đổi trạng thái toàn cục của trò chơi.
- **Observer Pattern (Implicit):** `WaveManager` đóng vai trò lắng nghe tín hiệu hoàn tất từ hệ thống `Augment` để biết khi nào nên bắt đầu đợt tấn công tiếp theo.

*Nhận xét kỹ thuật:* Việc tích hợp hệ thống `Augment` thông qua mối quan hệ lỏng với `PlantFactory` và `PlayScene` giúp trò chơi có chiều sâu chiến thuật cao. Cách thiết kế này cho phép dễ dàng thêm vào các loại `Augment` mới chỉ bằng cách mở rộng logic trong phương thức `applyAugmentEffect()` mà không cần cấu trúc lại toàn bộ `Game Loop`.

## 9 Hệ thống Quản lý Tài nguyên (Sun Management System)

Hệ thống Mặt trời (Sun System) đóng vai trò là nền tảng kinh tế của trò chơi, điều phối việc sản sinh tài nguyên tự nhiên và quản lý ngân sách để thực hiện các hành động mua thực thể Plant.

### 9.1 1. Bộ điều phối sản sinh tài nguyên (SunSpawner)

Lớp `SunSpawner` hoạt động như một bộ tạo tài nguyên tự động theo thời gian (Time-based Factory).

- **Quản lý nhịp độ:** Sử dụng biến `lastSpawn` kiểu `long` để theo dõi thời điểm cuối cùng tài nguyên được tạo ra, đảm bảo tính cân bằng cho gameplay.
- **Cơ chế thực thi:** Phương thức `update()` liên tục kiểm tra điều kiện thời gian. Khi đạt ngưỡng, nó sẽ khởi tạo thực thể `FallingSun` thông qua quan hệ phụ thuộc (*instantiates*).
- **Tương tác với Scene:** `SunSpawner` gọi hàm `addObject()` của `PlayScene` để đưa tài nguyên vào thế giới game tại các tọa độ ngẫu nhiên hoặc cố định.

### 9.2 2. Bộ quản lý ngân sách trung tâm (SunManager)

`SunManager` là thành phần chịu trách nhiệm lưu trữ và xử lý các giao dịch tài chính trong màn chơi.

- **Trạng thái tài nguyên:** Biến `sun` kiểu `int` lưu trữ số lượng mặt trời hiện có của người chơi.
- **Logic giao dịch:**
  - `add(amount)`: Tăng quỹ tài nguyên khi người chơi thu thập `FallingSun`.
  - `hasEnough(cost)`: Hàm kiểm tra logic (Boolean) giúp các hệ thống khác (như `SeedPacket`) biết người chơi có đủ khả năng chi trả hay không.
  - `spend(cost)`: Trừ số lượng tài nguyên sau khi xác nhận hành động mua cây thành công.

### 9.3 3. Thực thể Tài nguyên động (FallingSun)

Lớp `FallingSun` kế thừa từ `Actor`, đại diện cho đơn vị tài nguyên vật lý trên màn hình.

- **Hành vi:** Đóng gói logic di chuyển (rơi từ trên xuống) và phản hồi khi người chơi click chuột để thu thập.
- **Mối liên hệ:** Được điều khiển vòng đời bởi `SunSpawner` và khi bị tiêu hủy sẽ gửi tín hiệu cập nhật giá trị đến `SunManager`.

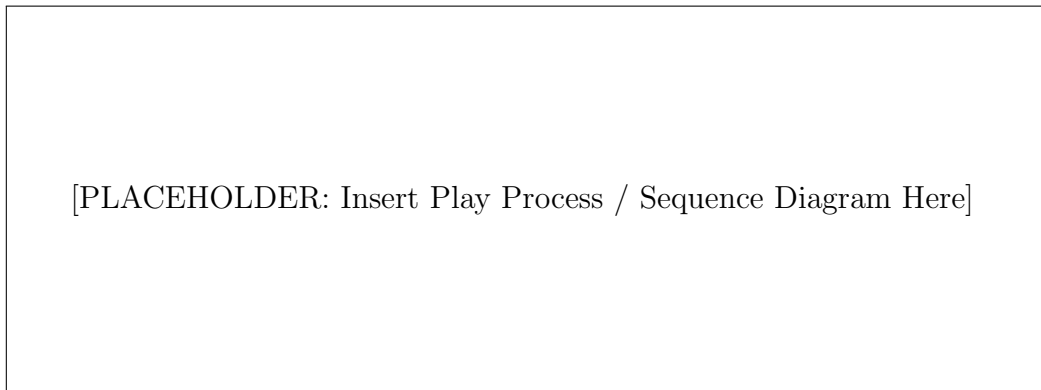
## 9.4 4. Mẫu thiết kế áp dụng trong Module Tài nguyên

- **Delegation (Ủy thác):** PlayScene không trực tiếp quản lý logic sinh mặt trời mà ủy thác toàn bộ cho SunSpawner, giúp mã nguồn của Scene chính gọn gàng và dễ bảo trì.
- **Encapsulation (Đóng gói):** Toàn bộ logic kiểm tra và trừ tiền được đóng gói bên trong SunManager, ngăn chặn việc thay đổi giá trị sun một cách tùy tiện từ các lớp bên ngoài.

*Nhận xét kỹ thuật:* Việc tách biệt giữa SunSpawner (logic tạo) và SunManager (logic lưu trữ) cho phép hệ thống dễ dàng mở rộng. Ví dụ, ta có thể thêm các loại mặt trời khác (mặt trời lớn, mặt trời từ cây hướng dương) mà không cần thay đổi cấu trúc của bộ quản lý ngân sách hiện tại.

## 4. UML (Architecture and Dependencies)

### 4.1. Play Process Diagram



Hình 1: Sequence diagram illustrating the game update and render loop.

### 4.2. Key Architectural Dependencies

- **Game Management:** The `GameScreen` manages the `GameWorld`.
- **Coordination:** The `GameWorld` coordinates all *Systems*.
- **Abstraction:** Controllers depend on `IGameSpawner`.



## 5. SOLID Principles

The design of the PVZ clone adheres to the **SOLID** principles to ensure maintainability, flexibility, and scalability.

### Single Responsibility Principle (SRP)

- **Explanation:** Each class or system should have only one reason to change.
- **Evidence:** Each `System` (e.g., `MovementSystem`, `CollisionSystem`) handles a single, distinct aspect of the game logic, operating only on the relevant components.

### Open/Closed Principle (OCP)

- **Explanation:** Software entities should be open for extension but closed for modification.
- **Evidence:** Adding a new plant type (e.g., a *Freezing Sunflower*) only requires creating a new `Plant` subclass and combining existing or new `Components`, without modifying existing core systems (e.g., `RenderSystem`, `PlantAttackSystem`).

### Liskov Substitution Principle (LSP)

- **Explanation:** Subtypes must be substitutable for their base types without altering the correctness of the program.
- **Evidence:** All subclasses of `Plant` or `Zombie` can be treated generically within the Systems loop. Their specialized behaviors are defined via `Components`, not by overriding fundamental base class behavior.
- **Code Example:**

Listing 1: LSP Implementation in Entity Processing

```
for (Entity plant : world.getEntitiesByGroup("plants")) {
    // Any subclass of Plant (Peashooter, Sunflower)
    // works perfectly here without special checks.
    plant.getComponent(HealthComponent.class).takeDamage(amount);
}
```

### Interface Segregation Principle (ISP)

- **Explanation:** Clients should not be forced to depend on interfaces they do not use.
- **Evidence:** The use of small, specific interfaces like `IGameSpawner` and `ISunReceiver` ensures classes only implement the methods they require.

```
public interface ISunReceiver {
    void addSun(int amount);
}
```

## Dependency Inversion Principle (DIP)

- **Explanation:** High-level modules should not depend on low-level modules; both should depend on abstractions.
- **Evidence:** The `GameWorld` (high-level) coordinates the `Systems` (low-level) via interfaces, rather than concrete implementations. The `HudController` depends on the `ISunReceiver` abstraction.

## Summary of SOLID Application

| Principle  | Where Applied              | Benefit                                              |
|------------|----------------------------|------------------------------------------------------|
| <b>SRP</b> | Entity Systems             | Clear separation of concerns; easier maintenance.    |
| <b>OCP</b> | Plant/Zombie subclasses    | Simplified extension; faster content addition.       |
| <b>LSP</b> | Entity handling in Systems | Safe polymorphism; reliable entity processing.       |
| <b>ISP</b> | IGameSpawner, ISunReceiver | Decoupled client logic; reduced interface bloat.     |
| <b>DIP</b> | GameWorld coordination     | Flexible architecture; easy to swap implementations. |

Bảng 2: Summary of SOLID principles application.

## 6. Design Patterns (Mô hình thiết kế)

Dự án sử dụng các mô hình thiết kế chuẩn để giải quyết vấn đề quản lý thực thể phức tạp trong môi trường Greenfoot, đặc biệt là sự kết hợp giữa lối chơi thủ thành và cơ chế Auto-battler.

### 6.1. Factory Method Pattern (Khởi tạo Plant/Zombie)

- **Ứng dụng:** Tách biệt logic khởi tạo các loại cây (`Peashooter`, `Repeater`, `GatlingPea`) khỏi logic trò chơi chính.
- **Minh chứng:**
  - Interface `IPlantFactory` định nghĩa các phương thức `createPlant` và `createSeedPacket`.
  - `PlantFactory` sử dụng `PlantType` (Enumeration) để quyết định thực thể cụ thể nào sẽ được tạo ra, giúp che giấu logic khởi tạo phức tạp bên trong.
- **Lợi ích:** Dễ dàng thêm loại cây mới mà không cần sửa đổi mã nguồn ở những nơi sử dụng Factory (`Open/Closed Principle`).

### 6.2. Observer Pattern (Hệ thống Event Bus))

- **Ứng dụng:** Quản lý các sự kiện trong trò chơi (chết, va chạm, hợp nhất) mà không làm các lớp bị phụ thuộc chặt chẽ (*tightly coupled*).
- **Minh chứng:**
  - Lớp `PlantEventBus` đóng vai trò là bộ điều phối trung tâm.
  - Các đối tượng quan tâm (ví dụ: trình quản lý âm thanh hoặc hiệu ứng) sẽ `subscribe()` vào `IPlantEventListener`.
  - Khi một cây chết, `publishDeath()` được gọi và tất cả các listener sẽ nhận được thông báo để xử lý logic riêng (phát âm thanh, xóa khỏi danh sách).
- **Lợi ích:** Tăng tính mô-đun hóa; hệ thống âm thanh và hệ thống logic có thể phát triển độc lập.

### 6.3. State Pattern (Hành vi Zombie/Plant)

- **Ứng dụng:** Quản lý hành vi của Zombie và Plant dựa trên trạng thái hiện tại của chúng.
- **Minh chứng:**
  - Interface `IZombieState` với các lớp cụ thể: `WalkingState`, `EatingState`, `DeadState`.
  - Lớp `Zombie` giữ một biến `currentState`. Khi Zombie gặp một cái cây, nó tự chuyển từ `WalkingState` sang `EatingState` thông qua phương thức `setState()`.
  - Mỗi trạng thái tự quản lý hoạt ảnh (*Animation*) và logic cập nhật riêng trong phương thức `update()`.
- **Lợi ích:** Loại bỏ các cấu trúc `if-else` hoặc `switch-case` cồng kềnh trong hàm `act()`, giúp mã nguồn dễ đọc và bảo trì.

## 6.4. Flyweight Pattern (Sprite Cache - Tối ưu bộ nhớ)

- **Ứng dụng:** Tối ưu hóa bộ nhớ khi xử lý một lượng lớn hình ảnh hoạt ảnh (*Sprites*).
- **Minh chứng:**
  - Lớp `SpriteCache` sử dụng một `HashMap<String, GreenfootImage[]>` để lưu trữ các mảng hình ảnh đã nạp.
  - Thay vì mỗi đối tượng `Zombie` nạp lại 20 tấm ảnh khi khởi tạo, chúng chỉ nhận tham chiếu đến mảng ảnh đã có sẵn trong `Cache`.
- **Lợi ích:** Giảm đáng kể dung lượng RAM sử dụng và tăng tốc độ khởi tạo thực thể khi trò chơi có nhiều đối tượng trên màn hình.

## 6.5. Strategy Pattern (Weighted Random Strategy)

- **Ứng dụng:** Sử dụng để quản lý logic tỉ lệ xuất hiện của các thẻ bài (*Rarity*) một cách linh hoạt.
- **Minh chứng:**
  - Lớp `RarityEntry` đóng vai trò là một chiến lược cấu hình, chứa cặp dữ liệu: loại thực thể (`packetClass`) và trọng số (`weight`).
  - `RupButton` nắm giữ một mảng `weightedPool` (tập hợp các chiến lược tỉ lệ). Khi người chơi nâng cấp (`Rup`), hệ thống không thay đổi code mà chỉ thay đổi cấu trúc của `weightedPool` để cập nhật tỉ lệ mới.
- **Lợi ích:** Tách biệt thuật toán tính xác suất khỏi đối tượng thực thi, cho phép dễ dàng điều chỉnh cân bằng game (game balancing) chỉ bằng cách thay đổi trọng số.

## 6.6. Command Pattern (Đối tượng Merger điều khiển việc hợp nhất)

- **Ứng dụng:** Đóng gói hành động hợp nhất giữa hai thực thể cây thành một đối tượng độc lập (*Merger*) để quản lý quá trình di chuyển và thực thi theo thời gian.
- **Minh chứng:**
  - Lớp `Merger` lưu trữ tham chiếu đến `source` (cây di chuyển) và `target` (cây đứng yên).
  - Phương thức `update()` đóng vai trò thực thi lệnh dần dần qua từng khung hình. Nếu quá trình bị ngắt quãng, phương thức `cancel()` sẽ được gọi để khôi phục trạng thái.
- **Lợi ích:** Tách biệt thời điểm "yêu cầu hợp nhất" và thời điểm "thực hiện hợp nhất", cho phép tạo các hiệu ứng hình ảnh mượt mà trước khi thay đổi logic dữ liệu.

## 6.7. Template Method Pattern (Khung thuật toán trong lớp Plant cha)

- **Ứng dụng:** Định nghĩa khung thuật toán cho việc xử lý di chuyển và hợp nhất trong lớp trừu tượng Plant.
- **Minh chứng:**
  - Lớp trừu tượng Plant định nghĩa phương thức `handleMergeMovement()`.
  - Các lớp cây con (`Peashooter`, `Sunflower`) kế thừa và sử dụng khung này, đảm bảo mọi loại thực thể đều tuân thủ đúng quy trình kiểm tra va chạm và kết thúc của hệ thống.
- **Lợi ích:** Tái sử dụng mã nguồn tối đa và đảm bảo tính nhất quán trong hành vi của các thực thể khác nhau.

## 6.8. Facade Pattern (PlayScene là cửa ngõ giao tiếp giữa các Actor và hệ thống Manager)

- **Ứng dụng:** Sử dụng PlayScene như một giao diện đơn giản hóa để AugmentCard tương tác với các hệ thống con phức tạp bên dưới.
- **Minh chứng:**
  - Thay vì AugmentCard phải truy cập trực tiếp vào các hệ thống quản lý lưới (`Grid`), quản lý bài (`SeedBank`), nó chỉ gọi các phương thức cấp cao thông qua PlayScene như `increasePlantSlots()` hoặc `getSunManager()`.
- **Lợi ích:** Giảm thiểu sự hiểu biết của các đối tượng Actor về cấu trúc nội bộ của thế giới trò chơi, giúp mã nguồn sạch sẽ và dễ bảo trì hơn.

## 7. Luồng trò chơi (Game Flow)

Trò chơi vận hành dựa trên sự phối hợp giữa lớp trung tâm `PlayScene` và các trình quản lý (*Managers*) chuyên biệt. Vòng lặp chính được điều khiển bởi phương thức `act()` của `Greenfoot`, chia thành các giai đoạn logic sau:

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Khởi tạo (Init)</b>          | <p>Khi <code>PlayScene</code> được tạo, thế giới trò chơi (<math>1111 \times 698</math>) được thiết lập với hình nền <code>maptft2.png</code>. Các thành phần cốt lõi được khởi tạo và đưa vào thế giới:</p> <ul style="list-style-type: none"> <li>• <b>Cơ sở hạ tầng:</b> <code>GridManager</code> (quản lý ô lưới), các máy cắt cỏ (<code>Lawnmower</code>) tại tọa độ cố định, và <code>hitbox</code> để theo dõi chuột.</li> <li>• <b>Giao diện (UI):</b> <code>SeedBank</code> (thanh thẻ bài), <code>SunDisplay</code> (hiển thị mặt trời), <code>Shovel</code> (xẻng), cùng các nút chức năng <code>RollButton</code> và <code>RupButton</code>.</li> <li>• <b>Hệ thống quản lý:</b> <code>SunManager</code>, <code>UpgradeManager</code>, và <code>MusicController</code>.</li> </ul> |
| <b>Xử lý tương tác</b>          | <p>Trong mỗi vòng lặp <code>act()</code>, hệ thống kiểm tra:</p> <ul style="list-style-type: none"> <li>• <b>Phím hệ thống:</b> Lắng nghe phím "escape" để thực hiện đóng/mở <code>SettingsMenuPanel</code> và điều trạng thái tạm dừng (<code>isPaused</code>).</li> <li>• <b>Chuột:</b> Cập nhật vị trí của <code>hitbox</code> liên tục theo tọa độ chuột để hỗ trợ việc tương tác và đặt cây.</li> <li>• <b>Cơ chế Roll:</b> Khi người dùng tương tác với <code>RollButton</code>, hệ thống chi trả 25 mặt trời để làm mới <code>SeedPacket</code> dựa trên tỉ lệ hiếm (<i>Rarity</i>) trong <code>RupButton</code>.</li> </ul>                                                                                                                                                            |
| <b>Cập nhật hệ thống</b>        | <p>Nếu trò chơi không ở trạng thái tạm dừng hoặc kết thúc, các bộ xử lý sau sẽ hoạt động:</p> <ul style="list-style-type: none"> <li>• <b>Wave &amp; Sun:</b> <code>Level</code> (<code>WaveManager</code>) kiểm tra và tạo các đợt <code>Zombie</code>; <code>SunSpawner</code> tạo tài nguyên mặt trời định kỳ.</li> <li>• <b>Hợp nhất (Merge):</b> <code>updateMergers()</code> xử lý tiến trình nâng cấp cây khi người chơi đặt các thực thể giống nhau gần nhau.</li> <li>• <b>Âm thanh:</b> <code>MusicController</code> cập nhật nhạc nền tương ứng với trạng thái trận đấu.</li> </ul>                                                                                                                                                                                                 |
| <b>Hiển thị &amp; Thứ tự vẽ</b> | <p>Trò chơi sử dụng phương thức <code>applyDefaultPaintOrder()</code> để quản lý lớp hiển thị (Z-index), đảm bảo các thành phần UI (như <code>SettingsMenu</code>) luôn nằm trên các thực thể game (<code>Zombie</code>, <code>Plant</code>). <code>drawWaveUI()</code> vẽ trực tiếp thông tin đợt tấn công lên màn hình.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Kiểm tra Win/Loss</b>        | <p><code>WinLossHandler</code> liên tục đánh giá điều kiện kết thúc:</p> <ul style="list-style-type: none"> <li>• <i>Thua cuộc:</i> Được xác định qua phương thức <code>hasLost()</code> khi bất kỳ <code>Zombie</code> nào vượt qua tọa độ <math>x &lt; 155</math>.</li> <li>• <i>Thắng cuộc:</i> Được xác định qua <code>hasWon()</code> khi <code>WaveManager</code> hoàn thành tất cả các đợt và không còn đối thủ.</li> </ul>                                                                                                                                                                                                                                                                                                                                                             |

*Quy trình: Input (Mouse/Key) → Manager Update → Merge Logic → Render (PaintOrder) → Win/Loss Check*

## 8. Conclusion + Future Work

Dự án **Plants vs Zombies Golden Spatula (Greenfoot)** đã chứng minh thành công việc áp dụng các nguyên lý **Lập trình hướng đối tượng (OOP)** cốt lõi trong phát triển trò chơi. Bằng cách tận dụng framework Greenfoot, dự án tập trung vào việc quản lý vòng đời đối tượng và tương tác trực quan giữa các thực thể thông qua cấu trúc phân cấp lớp chặt chẽ.

### Results Achieved:

- **Core Gameplay:** Xây dựng thành công một **World** tùy chỉnh với đầy đủ vòng lặp gameplay cơ bản của PVZ trên hệ thống tọa độ lưới lục giác.
- **Pure OOP Architecture:** Áp dụng triệt để tính *Kế thừa (Inheritance)* và *Đa hình (Polymorphism)*. Các thực thể như cây và zombie được tổ chức logic dưới các lớp cha (**Plant**, **Zombie**), giúp tối ưu hóa việc tái sử dụng mã nguồn.
- **Game Mechanics:** Triển khai hiệu quả hệ thống kinh tế mặt trời, cơ chế đặt cây và quản lý các đợt tấn công của zombie
- **System Integrity:** Tận dụng các công cụ của Greenfoot để xây dựng hệ thống phát hiện va chạm chính xác và cơ chế dọn dẹp đối tượng tối ưu.
- **Design Standards:** Tuân thủ nghiêm ngặt các nguyên tắc **SOLID**, tạo ra một cấu trúc project rõ ràng, dễ bảo trì và mở rộng



## 9. Links

### SOURCES:

[1] <https://github.com/rohangoel96/PlantsVsZombies-Game>

[2] <https://github.com/GeeeeekExplorer/PVZ>

[3] [https://github.com/Huy-IU-2k6/Plant\\_vs\\_Zombie](https://github.com/Huy-IU-2k6/Plant_vs_Zombie)

### OUR GITHUB CODES:

[4] <https://github.com/KietAnhCS/Plants-vs-Zombies>